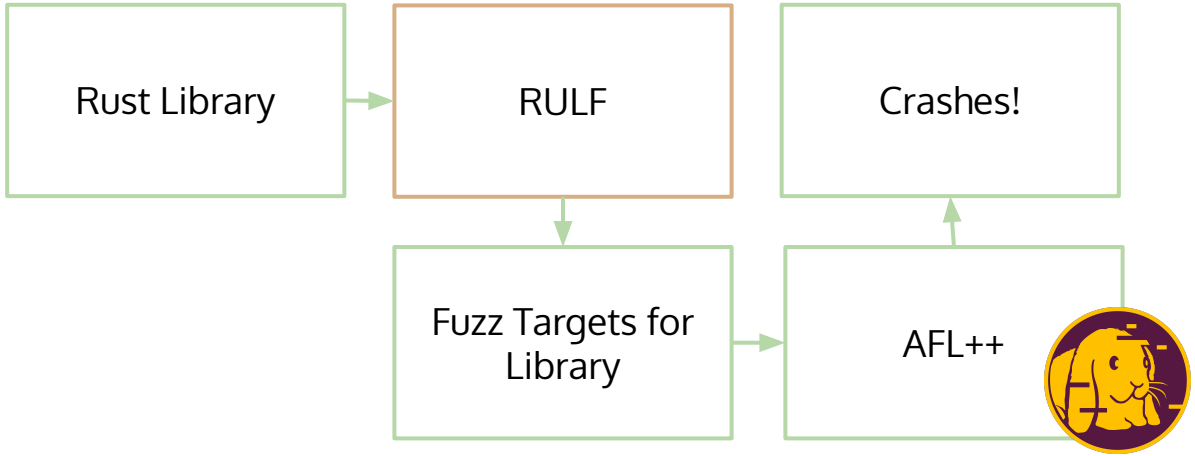
Paper review of ‘RULF: rust library fuzzing via API dependency graph traversal’

A presentation about the paper written by
Jianfeng Jiang, Hui Xu, and Yangfan Zhou

What did the authors build?

Fun Fact: AFL++ is presumably not named AFL++ because it's written in C++, even though it partially is, it's named that way because it's supposed to be a better version of the original, and now defunct AFL made by Michal Zalewski

They build a Rust Fuzz Target Generator that uses dependency graph traversal.



What is RULF?

Why RULF?

“Existing fuzzing tools are not directly applicable to library APIs due to the absence of fuzz targets.” [1]

A practical approach to ensure “robustness in Rust library development” [1] as the author’s put it, is to Fuzz these libraries, but doing so is too labor intensive because of the need for specific Fuzz targets to be generated by people.

What is a fuzz target and why is it important?

```
struct S1;
struct S2;
fn f1(a: i16) -> S1;
fn f2(b: u32) -> S2;
fn f3(c: &[u8]) -> S2;
fn f4(s1: S1, s2: &mut S2) -> S2;
fn f5(s2: &S2, d: &str);
```

(a) A toy Rust crate `toylib`.

```
fn fuzz_target_1_for_toylib(data: &[u8]) {
    if data.len() < 3 {return;}
    let a = to_i16(data, 0);
    let c = to_slice::<u8>(data, 2, data.len());
    let s1 = toylib::f1(a);
    let mut s2 = toylib::f3(c);
    let _ = toylib::f4(s1, &mut s2);
}
```

(b) Fuzz target 1.

Fuzzing works, why do we need to fuzz Rust?

Even though Rust is memory safe, bugs still exist in code bases.

“An interesting phenomenon of these bugs is that most of their host programs are libraries. Such bugs can only be triggered by composing a program with a specific usage of the library APIs.” [1]

The authors are talking about two specific lists of Rust bugs.

What kind of results did the authors get?

"[...] [W]e fuzz each target with a budget of 24 hours and **find 30 previously-unknown bugs** in seven libraries." [1]

What did the paper accomplish?

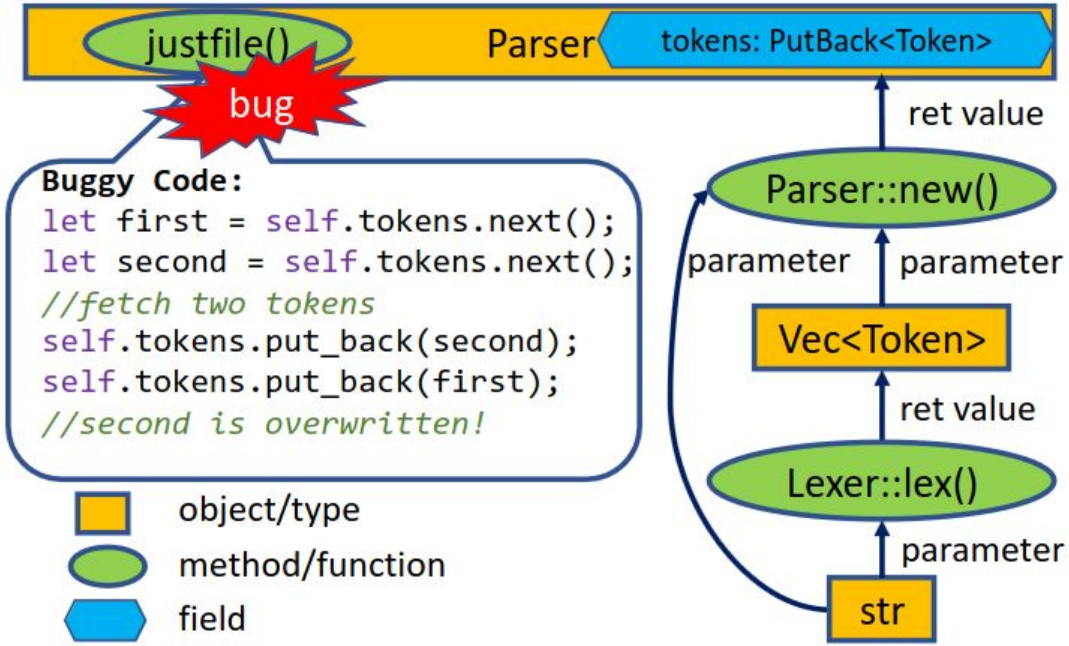
The authors list a few contributions that this paper brings to the table.

Firstly, a "pilot study" as the authors put it, for automatically generated fuzz targets for finding bugs in Rust Libraries. This is good news for anyone adding on to this area of research.

Secondly, they name their "sophisticated traversal algorithm" as this paper's second notable contribution. It's this algorithm that lets them have great API coverage with only "shallow" fuzz targets

The third thing the authors mention as being noteworthy from this paper is their implementation of this fuzzer, and they found 30 new bugs as previously mentioned

What type of bugs do you get in Rust?



How do they do this graph traversal?

Algorithm 1: BFS with pruning.

Input : A graph $G(FN, PAR, PE, CE)$,
Sequence length threshold max_len
Output: A set of sequences S_{BFS}

```
Init:
  S_BFS, S_new ← ∅ ;
  S_last ← {empty_sequence} ;
  i ← 1;
BFS with Pruning:
  while i ≤ max_len do
      for seq in S_last do
          if EndNodeTest(seq) then
              continue;
          for fn in G → FN do
              if ReachabilityTest(G, seq, fn) then
                  new_seq ← seq.append(fn);
                  S_new.add(new_seq);
      S_BFS.append(S_new);
      S_last ← S_new;
      S_new ← ∅;
      i ← i + 1 ;
  for seq in S_BFS do
      if RedundancyTest(seq) then
          S_BFS.remove(seq);
```

Dependency Graph

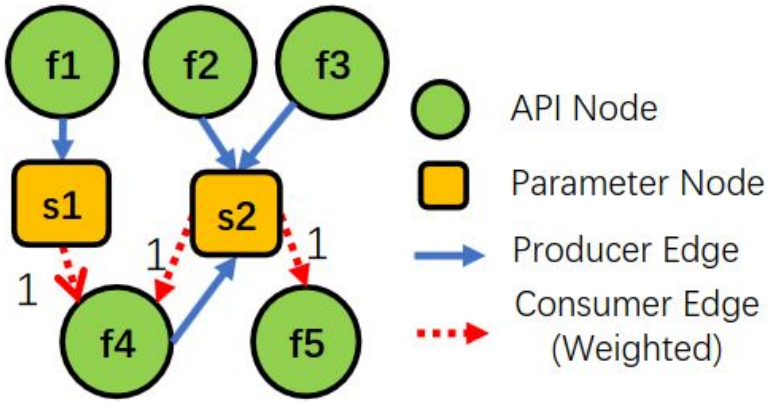


Fig. 3. An API dependency graph of crate `toylib`.

```
struct S1;
struct S2;
fn f1(a: i16) -> S1;
fn f2(b: u32) -> S2;
fn f3(c: &[u8]) -> S2;
fn f4(s1: S1, s2: &mut S2) -> S2;
fn f5(s2: &S2, d: &str);
```

RULF Workflow

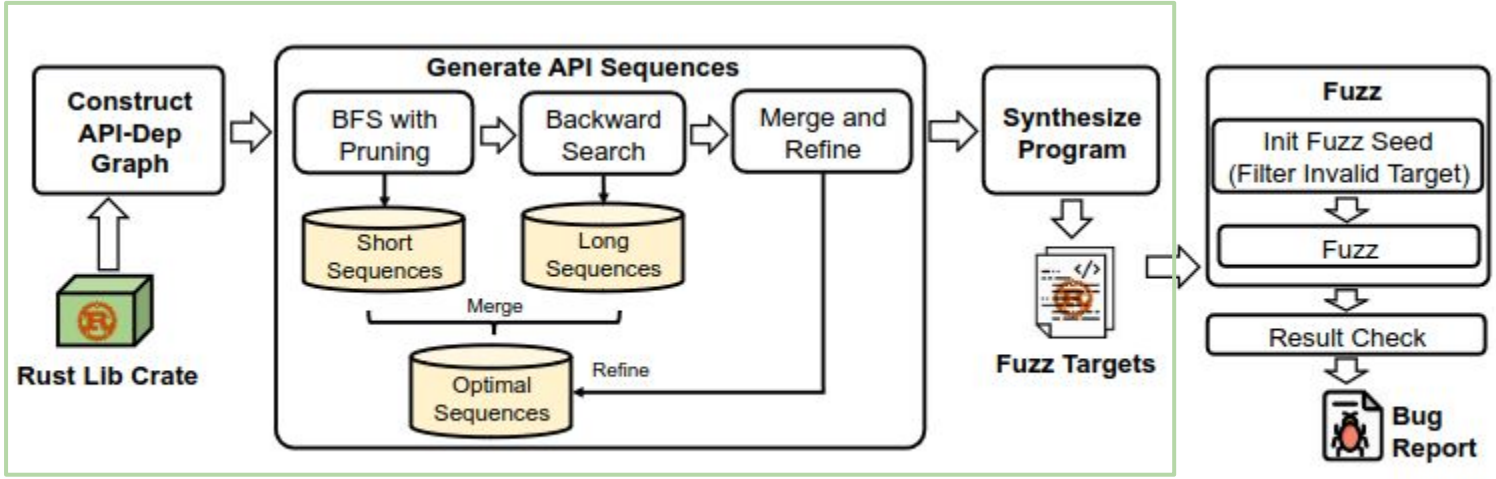


Fig. 4. Overview of our workflow.

Running Results

TABLE II

EXPERIMENT RESULTS ON 14 LIBRARIES. THE TOP ELEVEN ARE FROM CRATES.IO. THE LAST THREE ARE FROM GITHUB.

Crate name	version	API numbers	API coverage	Fuzz targets (invalid)	Finished targets	Average run time (hours)	Crashes (after cmin)	Bugs
clap	2.33.3	85	0.78	35(4)	27	3.24	236(28)	2
proc-macro2	1.0.24	60	0.65	33(1)	16	20.42	451(103)	0
serde_json	1.0.61	89	0.46	25	24	1.20	0(0)	0
http	0.2.3	74	0.35	19	16	4.42	0(0)	0
url	2.2.0	74	0.91	61	0	24	422(75)	2
regex-syntax	0.6.22	177	0.58	74	67	2.30	0(0)	0
regex	1.4.3	129	0.74	67	20	16.84	7274(257)	10
semver	0.11.0	12	0.92	7	0	24	147(1)	1
semver-parser	0.10.2	15	0.80	10	7	13.56	0(0)	0
flate2	1.0.19	31	0.68	11	11	0.35	2(1)	0
time	0.2.24	198	0.88	118	118	0.27	321(122)	7
bat	0.17.1	91	0.27	20	2	21.76	0(0)	0
xi-core-lib	0.4.0	212	0.27	34	34	0.53	34(10)	4
tui	0.13.0	188	0.37	48	48	3.12	105(39)	4

Types of Bugs Found

Crate	Range	Unwrap	Arith	UTF-8	Unreachable
clap	0	0	0	2	0
url	2	0	0	0	0
regex	6	0	2	2	0
tui	0	0	4	0	0
time	0	0	5	0	2
xi-core-lib	2	0	2	0	0
semver	0	1	0	0	0

"• Range: out-of-bound error
• Unwrap: call unwrap on Err() or None
• Arith: arithmetic overflow
• UTF-8: problems with UTF-8 strings
• Unreachable: visit codes with macro unreachable!()" [1]

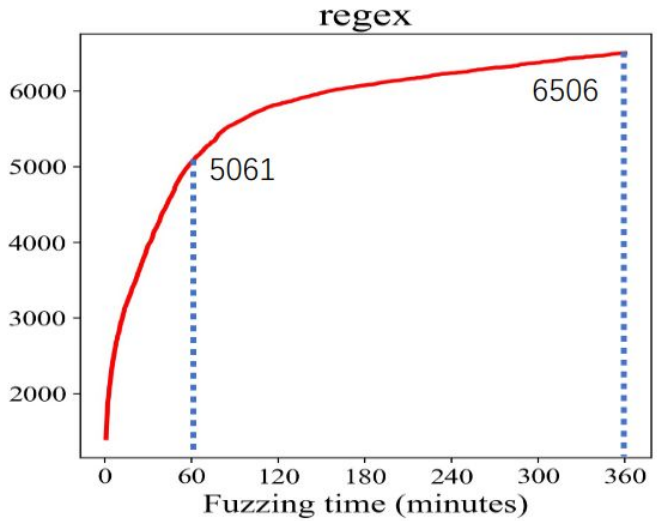
Different methods and their results

Crate	Random walk			FLC		
	Targets	API coverage	Bugs	Targets	API coverage	Bugs
url	61	0.34	1	18	0.32	1
regex	67	0.28	1	13	0.16	3
time	118	0.38	0	15	0.11	0

FLUT			RULF		
Targets	API coverage	Bugs	Targets	API coverage	Bugs
29	0.54	1	61	0.91	2
133	0.44	6	67	0.74	10
124	0.43	5	118	0.88	7

API Coverage

“RULF achieves API coverage ranging from 0.27 to 0.92.
The remaining APIs **cannot be covered** mainly because their parameters cannot be served by other APIs of the same crate.” [1]



Selected Research Questions

“RQ1: Can RULF meet the four objectives (i.e., validity, API coverage, efficiency, and effectiveness) of fuzz target generation?

RQ2: How does each step of our sequence generation algorithm help generate high-quality fuzz targets?

Can RULF outperform the state-of-the-art approach?

There is no available fuzz target generator on Rust now” [1]

Note, a tool called Fudge does exist for C++

Mutations

```rust
let s1 = toylib::f1(1);
let mut s2 = toylib::f2(2);
let _ = toylib::f4(s1, &mut s2); //f4 mutates s2
let _ = toylib::f5(&s2, "a"); //f5 uses the variable mutated by f4
```

Fig. 5. A mutation case on crate `toylib`.

Interesting & Relevant Citations

https://github.com/rust-fuzz -> https://github.com/rust-fuzz/arbitrary

```rust
use arbitrary::Arbitrary;

#[derive(Arbitrary)]
pub struct Rgb {
    pub r: u8,
    pub g: u8,
    pub b: u8,
}
```

Work Cited

[1] Jianfeng Jiang, Hui Xu, and Yangfan Zhou. 2022. RULF: rust library fuzzing via API dependency graph traversal. In Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering (ASE '21). IEEE Press, 581–592. https://doi.org/10.1109/ASE51524.2021.9678813